

Student Meal Planner

SWOT Analysis

A Personal Leadership Document for the Grubs4Scrubs Project

Date: ** [NOTE: Add the actual date you're filling this in.]

Author: Yao Cheng Zhou

LO mapped: LO6 Personal Leadership (Feedback and Reflection)

Context

I'm writing this toward the end of Semester 2 at Fontys FHICT. I've been working on Grubs4Scrubs (solo) and Media Bazar (group) simultaneously, while also self-studying C# through Codecademy and Microsoft Learn. The semester is wrapping up and I have a clearer picture now of where I stand than I did in week 3.

Date: [NOTE: Add the actual date you're filling this in.]

Strengths

- **I go deep on one thing at a time.** When I decided to learn ADO.NET properly, I didn't skim a tutorial and move on. I wrote every query by hand, debugged connection issues, and got to the point where I can write a repository from scratch without looking anything up. The RecipeRepository, UserRepository, and ShoppingItemRepository all came out of that focused effort.
- **I can build a full-stack feature end to end.** The shopping list is proof: I designed the SQL table, wrote the C# model, built the repository with ADO.NET, created the service layer, wired the controller, and then built the React page with inline editing. Same flow for the auth system (backend side). I'm not stuck in just frontend or just backend anymore.
- **I'm honest about what I don't know.** When I hit the JWT authentication system, I didn't pretend to understand it. I asked for explanations of every piece (claims, signing credentials, symmetric keys, token validation parameters) and didn't move on until each one made sense. That slowed me down but it means the code I wrote is actually mine.

[NOTE: Add or swap any of these for strengths you actually feel. These are based on what I've seen you do, but you know yourself better.]

Weaknesses

- **I start with the frontend when I should start with the backend.** Multiple times this semester I built a React page with hardcoded data, then had to rebuild parts of it when the API returned data in a different shape. The meal planner page still uses fake data because I built the UI first and haven't gotten to the API yet.

- **My commit habits get sloppy under pressure.** When a deadline is close, I stop writing proper commit messages and start pushing large, messy commits. Sprint 4 and 5 commits are probably bigger and less descriptive than sprint 1 and 2 commits. I know this is bad practice but I keep doing it.
- **I underestimate how long backend work takes.** ADO.NET boilerplate is slow to write. Every new entity means a new model, interface, repository, service, and controller. I keep planning sprint goals as if this takes an afternoon when it actually takes two or three days.

[NOTE: Edit these if they don't match your real experience. Weaknesses need to be honest or the SWOT is useless.]

Opportunities

- **The S4 Academic Preparation track.** If I perform well enough this semester, I can apply for the academic preparation track in Semester 4. That opens a path to a TU/e or JADS master's program, which is where I want to end up.
- **Grubs4Scrubs has real potential beyond the portfolio.** The gap analysis shows no one is building a meal planning app specifically for Dutch students on a budget. If I polish this after the semester, it could be a real side project or even a startup concept. The tech stack (React + .NET + SQL Server) is also very employable.
- **Self-study skills transfer.** The Codecademy and Microsoft Learn habits I built this semester aren't going away. I can apply the same approach to any new language or framework in future semesters.

[NOTE: Adjust these to what feels real. The S4 thing is based on what you've mentioned before.]

Threats

- **Time split between two projects.** Working on Grubs4Scrubs and Media Bazar at the same time means neither gets my full attention. Some weeks one project gets most of the effort and the other stalls. This is especially risky near deadlines.
- **Portfolio deadline pressure.** The documentation load for FHICT is heavy. Writing 15+ portfolio documents while also building features means something always gets rushed. Usually it's the docs.

- **Fontys teaching quality gaps.** Some concepts I needed (JWT auth, ADO.NET patterns, React state management patterns) weren't covered in class at all. I had to learn them entirely through self-study, which ate time I'd planned for feature development.

[NOTE: These are based on frustrations you've mentioned. Keep them factual and measured. A reflection with a real critique is stronger than one that pretends everything was fine.]

What I'm going to do about it

The biggest pattern here is the frontend-first habit. It costs me time every sprint. For the next submission (and for Semester 3), I'm committing to building the API endpoint and testing it in Swagger before touching the React page. That's a concrete change, not a vague "I'll be better at planning."

On commits, I'm going to try smaller, more frequent pushes even when a deadline is close. One commit per logical change. If that means 8 commits in a day instead of 1 big one, that's fine.

The time split between projects is harder to fix because it's structural, but I can at least block specific days for each project instead of switching back and forth within the same day. Context-switching is where the time disappears.

[NOTE: Write the real plan. What are you actually going to change based on what you noticed? This is the part that turns SWOT into LO6 evidence.]